

Convolutional networks

Dataset: Flowers102

How to use these notes

The primary text for this lecture is Géron, Chapter 12. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	What breaks when the image gets bigger	2
1.1	The brief	2
2	Derivation: weight sharing, equivariance, and memory	3
2.1	The parameter count, on the same image	3
2.2	What that assumption is, precisely	3
2.3	Equivariance is not invariance	4
2.4	Where the memory goes	5
3	Convolution, pooling, and an architecture	6
4	What it costs, and what it cannot do	7
4.1	Look at what it learned	8
5	The notebook	8
6	Exercises	8
	Summary	8

1 What breaks when the image gets bigger

Three lectures of neural networks, on flattened greyscale images. Flattening a 224 by 224 colour image gives 150,528 inputs, and a single dense layer of 1,000 units on that is 150 million parameters — for one layer, on one image size. Every one has to be learned from data, and none of them knows that two adjacent pixels are related.

Today's method fixes both problems at once, and the derivation says exactly how much it saves.

1.1 The brief

A horticultural wholesaler receives mixed pallets of cut stems and must sort them by species before the auction floor. One overhead camera per lane, one photograph per stem, and a human inspector who is the bottleneck.

In their words: *“Tell us which of our species it is. If you cannot tell, say so and we will look at it ourselves — but do not tell us the wrong one.”* Two requirements hiding in one sentence: a classifier over a fixed list, and a usable notion of confidence. We build the first today.

	Fashion-MNIST	Flowers102
Image	28 by 28, grey	variable, colour
Classes	10	102
Training images	60,000	1,020
Images per class	6,000	10
Subject fills the frame	always	sometimes

Ten times the classes and 59 times fewer labelled images. The split shipped with the dataset is 1,020 train, 1,020 val and 6,149 test — six times as many test images as training images, and the training split is balanced at exactly ten per class where the test split is not.

The standing constraint of applied vision

Images are cheap and labels are not. Somebody had to look at every photograph and name the species. Every application in this part of the course has more pixels than it can use and fewer labels than it wants — so a method that needs thousands of labelled examples per class is not a method you can deploy here.

What is hard here is position (the flower is not centred and not at a fixed scale), background, between-class similarity (several species separated by petal count and little else), and within-class variation. *Only the first has a structural answer*, and it is the one this chapter is about.

2 Derivation: weight sharing, equivariance, and memory

You replace a matrix multiplication with a convolution and the network becomes trainable. What exactly did you buy, and what did you give up? Three answers: a parameter count, a symmetry, and a memory bill that is not where anyone expects it.

2.1 The parameter count, on the same image

Fix the input and output shapes so the comparison is between two ways of computing the same sized thing: input 3 by 128 by 128, which is 49,152 values, and output 32 by 128 by 128, which is 524,288.

A dense layer connects every output to every input,

$$\text{\#weights} = n_{\text{in}}n_{\text{out}} = (3HW)(32HW) = 96H^2W^2,$$

which is 25,769,803,776 weights — 96 GB of float32 for one layer, before any gradient or optimiser state.

A convolution connects every output to a k by k window of every input channel and *uses the same weights at every position*,

$$\text{\#weights} = C_{\text{out}}C_{\text{in}}k^2 = 32 \cdot 3 \cdot 7 \cdot 7,$$

which is 4,704. Note that H and W do not appear: the image size is genuinely absent from the count. The ratio is H^2W^2/k^2 , which here is 5,478,275 times fewer weights for the same output shape — and with the 3 by 3 filters used everywhere else in the network, 29,826,162.

What the saving is not

It is *not* a saving in arithmetic. The convolution still computes $C_{\text{out}}HW$ outputs, each a sum over $C_{\text{in}}k^2$ terms — the same order of multiplications as the dense layer's, to within the window size.

What shrank is the number of *distinct numbers that have to be stored and learned*. The win is in memory and in statistics, not in floating-point operations, and confusing the two is how people end up expecting convolutions to be fast.

Weight sharing is a constraint, not a trick

A dense layer can represent any convolution: set its weight matrix to the block-Toeplitz matrix that repeats the kernel. Nothing is lost in expressive power.

The convolution is that matrix with its parameters *tied*. Tying them says a pattern means the same thing wherever it appears — true of photographs, false of a spreadsheet. This is Lecture 5's bias–variance trade made architecturally: a large reduction in variance, bought with an assumption we are prepared to defend.

2.2 What that assumption is, precisely

Let $T_{\mathbf{v}}$ shift an image by $\mathbf{v}$, so $(T_{\mathbf{v}}x)(\mathbf{p}) = x(\mathbf{p} - \mathbf{v})$. A map f is **equivariant** to translation when $f(T_{\mathbf{v}}x) = T_{\mathbf{v}}f(x)$ for every $\mathbf{v}$: shift then compute, or compute then shift, is the same answer.

Convolution is equivariant — one change of variable

Write $(w * x)(\mathbf{p}) = \sum_{\mathbf{u}} w(\mathbf{u})x(\mathbf{p} + \mathbf{u})$. Then

$$\begin{aligned}(w * T_{\mathbf{v}}x)(\mathbf{p}) &= \sum_{\mathbf{u}} w(\mathbf{u})x(\mathbf{p} + \mathbf{u} - \mathbf{v}) \\ &= (w * x)(\mathbf{p} - \mathbf{v}) = (T_{\mathbf{v}}(w * x))(\mathbf{p}).\end{aligned}$$

The proof works because w does not depend on $\mathbf{p}$ — which is exactly what weight sharing is. Untie the weights and the middle equality fails.

The equality is not exact everywhere. At the border, zero padding invents input that was not there, so a shift moves real content into invented content. Under stride greater than one, the output grid samples every s -th position, so equivariance survives only for shifts that are multiples of s . And in floating point, if the two computations sum the same terms in different orders. All three are why the next paragraph is a measurement rather than an assertion.

Shifting a trained layer's input by 16 pixels and dropping a 24-pixel border, the largest absolute difference on the interior is 0, against a largest activation of 4.15 in the same region. Bit-for-bit identical, not merely close — on this backend the two computations perform the same additions in the same order. Check it rather than assuming it, because on another backend it will not be exactly zero.

2.3 Equivariance is not invariance

A map is **invariant** when $g(T_{\mathbf{v}}x) = g(x)$: under a shift the output does not change, and the position is not kept. Invariance is strictly the stronger and strictly the lossier of the two, and it is what you get by discarding the equivariant structure.

Pooling manufactures it. A 2 by 2 max pool is invariant to any shift that keeps the maximum inside its own window; stack four stages and the tolerated shift grows to about eight pixels in the last feature map; a global pool over the whole map is invariant to every shift, because there is no position left to be sensitive to. The invariance is approximate and it is *built*, one factor of two at a time — nothing in the architecture declares it.

Cosine similarity, image against image shifted 16 pixels

the spatial feature maps, flattened	0.6379
after a global max pool	1.0000

The map changed by 36.2%; the pooled vector changed by 0.0004%.

Classification wants the second line: “this is a sunflower” is true wherever the sunflower is, so a representation that still carries the position is carrying a nuisance variable, and discarding it reduces variance at no cost in bias *provided the assumption really holds*.

Why per-pixel prediction cannot afford it

Segmentation asks, for every pixel, which class is it — and the answer *is* the position. Our input grid is 128 by 128 and the last feature map is 8 by 8, so one cell answers for 256 pixels: a 256-fold loss of spatial resolution, and two flower boundaries 15 pixels apart are

the same cell.

Lecture 14 has to put that resolution back, and the whole of its architecture is about how.

2.4 Where the memory goes

Your network has 4,807,494 parameters and your session died with *out of memory*. Which of those two facts caused the other?

What	Bytes each	Total
weights	4	18.3 MB
gradients, one per weight	4	18.3 MB
Adam's first moment	4	18.3 MB
Adam's second moment	4	18.3 MB
everything parameter-shaped	16	73.4 MB

Sixteen bytes per parameter, not four — the factor people forget, and still not the answer.

Lecture 10's derivation, in bytes: the reverse sweep evaluates each Jacobian *at the value the forward pass reached*, so every layer's output stays in memory from the moment it is computed until the backward pass consumes it. That is not an implementation detail of PyTorch; it is what reverse-mode automatic differentiation *is*.

Counted for the actual network, the stored activations are 5,964,646 floats per image — 22.8 MB. At batch 32 that is 728 MB: 9.9 times everything parameter-shaped put together, and about forty times the weights alone.

And it is not spread evenly. The activation cost of a layer is $C \times H \times W$, and channels double as the map quarters, so the total *halves* at every pooling stage: the first convolution's output is 64 MB at batch 32, the third 32 MB, the last 8 MB. 49% of the convolutional activation memory is in the first two layers — the layers with almost no parameters.

Parameters and activations live at opposite ends

Activations concentrate in the *early* layers, because H and W are still large. Parameters concentrate in the *late* layers, because $C_{\text{in}}C_{\text{out}}$ is large and k is fixed.

Two different bills, sent by two different halves of the same network — which is why “make the model smaller” and “make the run fit in memory” are different problems with different answers.

Checked against the allocator, the prediction by counting outputs is 728 MB and the measurement is 568. The measurement is *lower* because the allocator frees an activation as soon as nothing can still need it, and in a plain `Sequential` the ReLU outputs of already-consumed layers qualify. The prediction is the ceiling, which is the number you want when sizing a run.

When a run runs out of memory

Halve the batch, not the model. Activation memory is linear in the batch size and the parameter memory does not move at all, so the first lever costs you nothing but wall clock. In order: smaller batch, then smaller input resolution (quadratic), then gradient checkpointing,

then mixed precision.

Reducing the parameter count is near the bottom of that list — and on this network it would mostly delete the dense head, which is not where the memory is.

3 Convolution, pooling, and an architecture

A feature worth detecting in the top left corner is worth detecting in the bottom right, so use the same weights everywhere. That is the whole idea.

For an input with C channels and a kernel of size k by k ,

$$z_{ij}^{(f)} = b^{(f)} + \sum_{c=1}^C \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} w_{c,u,v}^{(f)} x_{c,i+u,j+v}$$

— a dot product between the kernel and a window, repeated at every position, with the sum over c being why a filter reads all input channels at once.

Three numbers describe it: the kernel size k , how much of the input one output sees; the stride s , how far the window moves; and the padding p , how many zeros ring the input. Then

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1,$$

and with $s = 1$ and $p = \lfloor k/2 \rfloor$ the output keeps the input's height and width.

The default nobody asked for

padding defaults to *zero*, which silently shrinks every feature map by $k - 1$ and will eventually give you a shape error twelve layers later. Reviewer question 5, every time.

Pooling halves the height and width by keeping the largest value in each 2 by 2 block. Three quarters of the activations disappear, and with them three quarters of the memory and arithmetic in every later layer; it has no parameters; and each later unit sees a larger piece of the original image. The fourth consequence is the invariance the derivation spent twenty minutes on.

Every convolution is followed by a batch normalisation, because ten layers of untreated convolutions on 1,020 images do not train — the loss sits at $\log 102 \approx 4.62$ and the network predicts one class. That is Lecture 11's $\rho \neq 1$ in a new architecture, and here, unlike there, initialisation alone does not reach it. Lecture 11 measured batch normalisation making a repaired network *worse* by 4.3 points; the size of an effect depends on what else is in the system.

Stage	Layers	Output
1	conv 7 by 7 $\times$ 32, conv 3 by 3 $\times$ 32, pool	32 by 64 by 64
2	conv $\times$ 64, conv $\times$ 64, pool	64 by 32 by 32
3	conv $\times$ 128, conv $\times$ 128, pool	128 by 16 by 16
4	conv $\times$ 256, pool	256 by 8 by 8
head	flatten, dense 256, dropout, dense 102	102

Part	Parameters	Share
Every convolution and batch-norm layer together	586,720	12.2%
One dense layer, 16,384 to 256	4,194,560	87.3%
Output layer, 256 to 102	26,214	0.5%
Total	4,807,494	100%

Nine parameters in ten are in the layer that is *not* convolutional. And 4,807,494 parameters against 1,020 training images is 4713 parameters per training image: there is nothing in the architecture stopping this network from storing the training set, and dropout at 0.5 is the only thing asked to prevent it.

The training loop is exactly Lecture 10's five lines. Convolution changed the model; it did not change the loop, because autograd does not care what the modules are.

4 What it costs, and what it cannot do

Learning rate	Mean test accuracy	sd over seeds
0.001, Adam's default	2.7%	1.31 pts
0.0003	14.7%	2.59 pts

12.1 points, from one keyword argument nobody wrote down. The default does not diverge — it plateaus low, which is the failure mode that does not announce itself.

Eighty epochs at batch 32 is 32 steps per epoch and 2,560 optimiser steps in total, 253 seconds for the seed-42 run. The training curve reaches 71.8% while the validation curve stops climbing early: 58 points of the training accuracy is memorisation, which is the shape Lecture 5 named as variance. The last two thirds of the wall clock bought almost nothing.

Model	Test accuracy	Wall clock
Uniform random guess	0.98%	—
Always the commonest species	3.87%	—
Convolutional network, from scratch	15.3%	364 s
What the operator needs	90%	—

Four times the majority baseline, and a long way from deployable.

Why five runs and not one

Seeds 42 to 46 give 11.0%, 15.7%, 18.1%, 16.1% and 15.7% — 7.1 points between best and worst with nothing changed but the seed.

One run of a network this size on 1,020 images is an anecdote, and any comparison you make from here that is smaller than 7.1 points is not a comparison.

4.1 Look at what it learned

The first layer's weights are 4,704 numbers as 32 filters of 7 by 7 by 3 — small enough to look at. Against the initialisation they are still almost the same vector — the mean absolute change is 23.9% of the mean weight, and 247 of the 4,704 changed sign.

The weights moved. *No structure appeared* — and 253 seconds of training bought that. Two reasons, both worth knowing. The gradient reaching this layer is small, because ten layers of chain rule separate it from the loss and 1,020 images give only 2,560 optimiser steps. And its *scale does not matter*: a batch norm follows immediately, so normalising the output makes the overall size of these weights irrelevant to the function, and there is no gradient pressure on it at all.

The textbook picture of a first layer — colour blobs, oriented edges — is real, and you will see it next lecture. It comes from a corpus a thousand times this size.

5 The notebook

What to change

Halve the batch from 32 to 16 and watch the allocator. The parameter figure does not move and the activation figure halves, which is the memory argument measured rather than read — and it is also the first lever the rule above tells you to pull.

6 Exercises

1. A dense layer and a convolution produce the same output shape from the same image. Give both parameter counts symbolically, and state which quantity is absent from the second. [5]
2. Prove that convolution is equivariant to translation, and name the property of the kernel the proof depends on. [4]
3. Explain why pooling's invariance helps classification and is fatal for per-pixel prediction. [4]
4. A training run fails with out-of-memory. The model has 4.8 million parameters. State which of parameters or activations you would investigate first, with the reason, and give the first lever you would pull. [5]
5. A first layer's filters look like noise after training. Give two reasons this can happen even when the network as a whole is learning. [4]

Summary

Weight sharing costs 5,478,275-fold fewer parameters than the dense layer with the same output shape — a saving in memory and in statistics, not in arithmetic. It is precisely the condition for translation equivariance, and the proof is one change of variable that depends on the kernel not

varying with position. Pooling trades that equivariance for invariance, which classification wants and per-pixel prediction cannot have: measured, the spatial maps of a shifted image agree at cosine 0.6379 and the globally pooled vectors at 1.0000.

The memory bill is not where anyone expects. Parameters and their optimiser state are 73.4 MB at sixteen bytes each; activations at batch 32 are 728 MB, concentrated in the early layers where there are almost no parameters. So halve the batch, not the model.

Trained from scratch on 1,020 images the network reaches 15.3% against a 3.87% majority baseline, with 7.1 points between the best and worst of five seeds. Its learning rate is worth 12.1 points, its training accuracy reaches 71.8% with 58 points of that being memorisation, and its first-layer filters are still noise — for two reasons, one of which is that a batch norm downstream removes any gradient pressure on their scale.